RÉPUBLIQUE TUNISIENNE
Ministère de l'Enseignement Supérieur
et de la Recherche Scientifique
Université de Carthage
Institut National des Sciences Appliquées et de Technologie

DevSecOps

Spécialité : Génie Logiciel
Niveau : 5e année

TP1 – SAST & SCA

Analyse Statique du Code et Sécurité des Dépendances

Sécurisation d'une application web Flask conteneurisée
Bandit, OWASP Dependency-Check, pip-audit et GitHub Actions

Présenté par :

Yassine Dahmoul
Mohamed Sahbi Ben Kalaia
Mohamed Dhia Medini
Takoua Ayadi

Encadré par : Mme Saloua Ben Yahia

Année Universitaire : 2026 – 2027

Table des matières

1 Contexte et Objectifs **2**

2 Environnement Technique **2**

3 Partie I – Mise en place de l'environnement **2**
3.1 Structure du Projet . . . 2
3.2 Application étudiée . . . 3
3.3 Conteneurisation de l'application . . . 3
3.4 Organisation du dépôt Git . . . 3

4 Partie II – Analyse Statique (SAST) avec Bandit **4**
4.1 Principe et lancement du scan . . . 4
4.2 Résultats initiaux . . . 4
4.3 Vulnérabilité 1 – Injection SQL (B608, CWE-89) . . . 4
4.4 Vulnérabilité 2 – Injection de commande OS (B602, CWE-78) . . . 5
4.5 Autres vulnérabilités corrigées . . . 6
4.6 Résultats après correction . . . 6
4.7 Limites observées du SAST . . . 7

5 Partie III – Analyse des Dépendances (SCA) **7**
5.1 Outils et commandes . . . 7
5.2 Résultats initiaux . . . 7
5.3 Dépendance vulnérable choisie : PyYAML 5.3 → 6.0.3 . . . 8
5.4 Autres mises à jour . . . 9
5.5 Durcissement de l'image Docker . . . 9
5.6 Résultats après mise à jour . . . 10
5.7 Comparaison des outils SCA . . . 10

6 Partie IV – Démonstration et Évaluation **10**
6.1 Scénario de démonstration . . . 10
6.2 Automatisation dans le pipeline CI (shift-left) . . . 11

7 Conclusion **11**

Annexes – Captures d'écran **12**

Annexe – Détail des vulnérabilités des dépendances **17**

1 Contexte et Objectifs

Ce TP a pour objectif de mettre en pratique l'approche **« shift-left »** en sécurité : détecter et corriger les vulnérabilités *dès l'écriture du code et le choix des bibliothèques*, plutôt qu'au moment d'un audit ou d'un test d'intrusion avant la mise en production. Plus une faille est découverte tard, plus elle coûte cher à corriger.

Deux familles d'outils automatisables sont utilisées :

Approche	**Ce qui est analysé**	**Outil utilisé**
SAST (*Static Application Security Testing*)	Notre code source, sans l'exécuter	Bandit
SCA (*Software Composition Analysis*)	Les bibliothèques tierces (directes et transitives)	OWASP Dependency-Check, pip-audit

Objectifs principaux :

— Conteneuriser une application web Python (Flask) avec Docker.

— Lancer un scan **SAST** avec Bandit, identifier et **corriger** au moins deux vulnérabilités.

— Lancer un scan **SCA** avec OWASP Dependency-Check, identifier au moins une dépendance vulnérable, la **mettre à jour** et justifier cette mise à jour.

— **Démontrer** que les scanners ne signalent plus les problèmes corrigés.

2 Environnement Technique

Élément	**Détail**
Conteneurisation	Docker Desktop 29.6.1
Langage / framework	Python, Flask (application « Notes App »)
SAST	Bandit 1.9.4 (image officielle `ghcr.io/pycqa/bandit/bandit`)
SCA	OWASP Dependency-Check 13.0.0 (image `owasp/dependency-check`)
SCA (contre-vérification)	pip-audit 2.10.1 (base PyPA / OSV)
Gestion de version	Git + dépôt GitHub `YassineDHML/tp1-devsecops-sast-sca`
CI/CD	GitHub Actions

Aucun outil d'analyse n'est installé sur la machine : tous sont exécutés dans des conteneurs Docker, ce qui rend les résultats reproductibles.

3 Partie I – Mise en place de l'environnement

3.1 Structure du Projet

```
tp1-devsecops-sast-sca/
 app/
   app.py                    # application Flask
   templates/                # vues Jinja2
 requirements.txt            # dependances figees (directes + transitives)
 Dockerfile
 .dockerignore  .gitignore
 scripts/scan.ps1            # lance SAST + SCA sur une version Git
```

```
.github/workflows/security.yml
reports/sast/                # rapports Bandit avant / apres
reports/sca/                 # rapports Dependency-Check et pip-audit avant /
   apres
```

3.2 Application étudiée

Notes App est une petite application Flask de prise de notes : inscription et connexion, création et recherche de notes, import de notes au format YAML, préférences utilisateur, et deux outils réseau (ping d'un hôte, récupération d'une URL). Sa **version initiale** reproduit des erreurs de développement courantes et utilise des dépendances anciennes (Flask 0.12, PyYAML 5.3, requests 2.19...), comme une application « legacy ».

3.3 Conteneurisation de l'application

L'image initiale est construite sur `python:3.7-slim`, installe les dépendances et lance le serveur de développement Flask.

Listing 1 – Dockerfile initial de l'application (version `v1.0-vulnerable`)

```
FROM python:3.7-slim

WORKDIR /app

RUN apt-get update && apt-get install -y iputils-ping && rm -rf /var/lib/apt/
    lists/*

COPY . .
RUN pip install -r requirements.txt

EXPOSE 5000
CMD ["python", "app/app.py"]
```

Listing 2 – Construction et lancement de l'image

```
docker build -t notes-app:vulnerable .
docker run -d --name notes-vuln -p 5001:5000 notes-app:vulnerable
```

Dès le démarrage, les journaux du conteneur affichent un premier signal d'alarme : `Debugger is active!` (Figure 1). Le débogueur interactif de Werkzeug est exposé sur toutes les interfaces.

3.4 Organisation du dépôt Git

L'historique est découpé pour que chaque étape du TP soit visible et comparable :

Commit / tag	Contenu
`v1.0-vulnerable`	Application initiale vulnérable
`SAST : correction...`	Corrections des vulnérabilités signalées par Bandit + rapports Bandit
`v2.0-secure`	Mise à jour des dépendances, image Docker durcie, rapports SCA
Commits suivants	Script de scan, pipeline CI, rapport

4 Partie II – Analyse Statique (SAST) avec Bandit

4.1 Principe et lancement du scan

Bandit parcourt l'arbre syntaxique (AST) de chaque fichier Python et applique des règles (`B1xx` à `B7xx`). Chaque problème reçoit une **sévérité** (impact s'il est exploitable) et un **niveau de confiance** (probabilité qu'il s'agisse d'un vrai problème).

Listing 3 – Lancement de Bandit via Docker

```
docker run --rm -v "${PWD}:/src" -w /src --entrypoint bandit `
    ghcr.io/pycqa/bandit/bandit -r app -f html -o reports/sast/bandit-before.
    html
```

4.2 Résultats initiaux

Le premier scan signale **15 problèmes : 4 High, 7 Medium et 4 Low**.

#	Règle	Sév. / Conf.	Ligne	Problème (CWE)
1	B608	Medium / Low	102	Injection SQL dans `/login` (CWE-89)
2	B608	Medium / Low	152	Injection SQL dans `/search` (CWE-89)
3	B602	**High** / High	199	Injection de commande OS, `shell=True` (CWE-78)
4	B704	Medium / High	157	XSS réfléchi : `Markup()` sur la saisie (CWE-79)
5	B301	Medium / High	189	Désérialisation `pickle` d'un cookie (CWE-502)
6	B403	Low / High	11	Import du module `pickle` (CWE-502)
7	B506	Medium / High	166	`yaml.load()` sur un fichier utilisateur (CWE-20)
8	B324	**High** / High	47	Mots de passe hachés en MD5 (CWE-327)
9	B105	Low / Medium	24	`secret_key` Flask en dur (CWE-259)
10	B105	Low / Medium	26	Mot de passe administrateur en dur (CWE-259)
11	B501	**High** / High	206	`requests` avec `verify=False` (CWE-295)
12	B113	Medium / Low	206	Requête HTTP sans timeout (CWE-400)
13	B201	**High** / Medium	213	`app.run(debug=True)` (CWE-94)
14	B104	Medium / Medium	213	Écoute sur `0.0.0.0` (CWE-605)
15	B404	Low / High	13	Import du module `subprocess` (CWE-78)

TABLE 1 – Résultats Bandit sur la version initiale

Le TP demande d'en corriger au moins deux. **Toutes les vulnérabilités High et Medium ont été corrigées**, ainsi que les secrets en dur. Les deux premières sont détaillées ci-dessous (Figures 2 et 3).

4.3 Vulnérabilité 1 – Injection SQL (B608, CWE-89)

Listing 4 – Code vulnérable de `/login`

```
query = "SELECT id, username FROM users WHERE username = '%s' AND password = '%s
    '" % (
    username,
    hash_password(password),
)
user = get_db().execute(query).fetchone()
```

Pourquoi c'est dangereux. La saisie de l'utilisateur est insérée directement dans le texte de la requête SQL. Des caractères SQL (apostrophe, commentaire…) peuvent alors **modifier la structure de la requête** au lieu d'être traités comme des données. Sur `/login`, un attaquant

peut ainsi se connecter sous l'identité d'un autre utilisateur sans connaître son mot de passe. Sur `/search`, il peut lire d'autres tables, par exemple les hachés de mots de passe.

Correction : requêtes paramétrées. La requête et les données sont transmises séparément à SQLite. La saisie est toujours traitée comme une valeur, jamais comme du code.

Listing 5 – Code corrigé de `/login` et `/search`

```
# [SAST B608] Requete parametree : la saisie n'est jamais concatenee au SQL.
user = get_db().execute(
    "SELECT id, username, password FROM users WHERE username = ?",
    (request.form["username"],),
).fetchone()
if user and check_password_hash(user["password"], request.form["password"]):
    ...

notes = get_db().execute(
    "SELECT id, title, content FROM notes WHERE owner_id = ? AND title LIKE ?",
    (session["user_id"], "%" + q + "%"),
).fetchall()
```

Remarque : Bandit attribue à B608 une confiance *Low*. Il repère un motif (une chaîne SQL formatée) sans pouvoir prouver que la variable provient de l'utilisateur. Ici, c'est bien un **vrai positif**. L'analyse SAST demande donc toujours une vérification humaine (triage).

4.4 Vulnérabilité 2 – Injection de commande OS (B602, CWE-78)

Listing 6 – Code vulnérable de `/tools/ping`

```
host = request.args.get("host", "127.0.0.1")
output = subprocess.check_output("ping -c 1 " + host, shell=True, stderr=
    subprocess.STDOUT)
```

Pourquoi c'est dangereux. Avec `shell=True`, la chaîne est interprétée par `/bin/sh`. Les métacaractères du shell (`;`, `&&`, `|`…) présents dans `host` permettent d'enchaîner **n'importe quelle commande** sur le serveur. Dans l'image initiale, le processus tourne en **root** et la route n'exige aucune authentification : c'est une exécution de code à distance (RCE).

Correction : validation stricte et suppression du shell. La saisie est d'abord validée comme adresse IP, puis la commande est lancée sans shell :

Listing 7 – Code corrigé de `/tools/ping`

```
@app.route("/tools/ping")
@login_required
def ping():
    # [SAST B602] Validation stricte de l'entree (adresse IP uniquement) et
    # appel sans shell, avec une liste d'arguments et un chemin absolu.
    try:
        ip = ipaddress.ip_address(request.args.get("host", "127.0.0.1"))
    except ValueError:
        abort(400, "Adresse IP invalide")
    result = subprocess.run(
        ["/bin/ping", "-c", "1", str(ip)], capture_output=True, timeout=5, check
   =False
    )
```

1. **Liste blanche** : seule une adresse IPv4/IPv6 valide est acceptée ; tout le reste renvoie une erreur 400 (Figure 13).
2. **Pas de shell** : la commande est une liste d'arguments, aucun métacaractère n'est interprété.
3. **Chemin absolu** `/bin/ping` (évite un détournement par le `PATH`), **timeout**, **authentification** obligatoire et, dans l'image, utilisateur **non-root**.

4.5 Autres vulnérabilités corrigées

Règle	Risque	Correction apportée
B704 (XSS)	`Markup()` désactive l'échappement : du JavaScript peut s'exécuter dans le navigateur de la victime	Suppression de `Markup()` ; échappement automatique de Jinja2 (Figure 11) ; en-tête CSP
B301 / B403 (pickle)	Désérialiser un cookie contrôlé par le client revient à exécuter du code arbitraire	Préférence stockée dans la session Flask (JSON signé HMAC) + liste blanche des valeurs
B506 (`yaml.load`)	Instanciation d'objets Python arbitraires ; combiné à PyYAML 5.3 : RCE (§5.3)	`yaml.safe_load()` + validation de la structure + taille d'upload limitée à 1 Mo
B324 (MD5)	Hachés cassables en quelques secondes en cas de fuite de la base	`generate_password_hash()` de Werkzeug (scrypt, sel aléatoire)
B105 (secrets en dur)	`secret_key` connue : forge de sessions ; secrets visibles dans l'historique Git	Variables d'environnement `SECRET_KEY` et `ADMIN_PASSWORD` ; clé aléatoire par défaut
B501 / B113	Interception TLS (MITM) ; blocage des workers (DoS) ; SSRF vers le réseau interne	`verify=True`, `timeout=5`, pas de redirection, seules les URL publiques en http(s) sont acceptées
B201 / B104	Débogueur Werkzeug exposé, qui permet d'exécuter du Python	`debug` piloté par `FLASK_DEBUG`, écoute sur `127.0.0.1`, **gunicorn** en production

Table 2 – Synthèse des autres corrections SAST

4.6 Résultats après correction

Sévérité	Avant	Après
High	4	**0**
Medium	7	**0**
Low	4	2
Total	15	2

Table 3 – Bandit avant / après correction

Les **deux problèmes Low restants** (Figure 4) ont été analysés et **acceptés** :

— **B404** (import de `subprocess`) : avertissement générique, le module est nécessaire à la fonctionnalité ping.

— **B603** (`subprocess` sans shell) : Bandit rappelle de vérifier les entrées. C'est fait : pas de shell, chemin absolu, argument validé par `ipaddress`.

Ces lignes n'ont volontairement **pas** été masquées par `# nosec`, par souci de transparence. En CI, le seuil de blocage est fixé à *Medium*.

4.7 Limites observées du SAST

— **Faux positifs possibles** : les règles à confiance *Low* demandent une confirmation humaine.

— **Faux négatifs** : Bandit ne raisonne pas sur la logique applicative. Il n'a pas détecté la SSRF de `/tools/fetch`, l'absence d'authentification des outils réseau ni l'absence de protection CSRF. Ces points ont été trouvés à la relecture.

— Bandit n'analyse ni les templates, ni le Dockerfile, ni les dépendances : c'est le rôle de la SCA.

5 Partie III – Analyse des Dépendances (SCA)

5.1 Outils et commandes

OWASP Dependency-Check identifie chaque composant, lui associe un identifiant **CPE** puis recherche les **CVE** correspondantes dans la base **NVD** du NIST.

Listing 8 – Lancement d'OWASP Dependency-Check

```
docker run --rm -v "${PWD}:/src" -v dc-data:/usr/share/dependency-check/data `
  owasp/dependency-check:latest `
  --project "TP1 Notes App" --scan /src --exclude "**/reports/**" --exclude
    "**/.git/**" `
  --enableExperimental --disableOssIndex `
  --nvdDatafeed "https://dependency-check.github.io/DependencyCheck_Builder/
    nvd_cache/" `
  --format HTML --format JSON --out /src/reports/sca
```

— `--enableExperimental` : l'analyseur de `requirements.txt` est expérimental. Sans cette option, aucune dépendance Python n'est détectée.

— `--nvdDatafeed` : sans clé d'API NVD, le téléchargement par l'API est très limité en débit. On utilise le miroir du projet Dependency-Check (environ 100 s au premier lancement), conservé ensuite dans le volume `dc-data`.

— `--disableOssIndex` : l'analyseur Sonatype OSS Index demande désormais un jeton personnel.

pip-audit (outil de la PyPA) sert de **contre-vérification**. Il identifie les paquets par leur nom PyPI exact et interroge la base OSV / PyPI Advisory :

Listing 9 – Lancement de pip-audit

```
docker run --rm -v "${PWD}:/src" -w /src python:3.13-slim sh -c `
  "pip install -q pip-audit && pip-audit -r requirements.txt --no-deps --disable
    -pip"
```

5.2 Résultats initiaux

Analyse	Cible	Résultat
Dependency-Check	`requirements.txt`	**4 CVE** (3 Critical, 1 High)
Dependency-Check	Paquets installés dans l'image Docker	**44 alertes** (7 Critical, 12 High)
pip-audit	`requirements.txt`	**47 vulnérabilités** dans 9 paquets

Table 4 – Résultats SCA sur la version initiale (Figures 5 et 6)

Paquet	Version	Nb	Vulnérabilité la plus marquante (CVSS NVD)	Retenue
PyYAML	5.3	2	CVE-2020-1747 / CVE-2020-14343 : **9.8 Critical**, exécution de code	**6.0.3**
Werkzeug	0.14.1	12	CVE-2022-29361 : 9.8 (*request smuggling*) ; CVE-2019-14806 : 7.5 (PIN du débogueur)	3.1.9
certifi	2018.4.16	2	CVE-2023-37920 : 9.8 (autorités racines non fiables)	2026.7.22
requests	2.19.1	5	CVE-2018-18074 : 7.5 (fuite de l'en-tête `Authorization`)	2.34.2
urllib3	1.23	12	CVE-2023-43804 (fuite de cookies lors de redirections)	2.8.0
Jinja2	2.10	6	CVE-2019-10906, CVE-2025-27516 (évasions du *sandbox*)	3.1.6
Flask	0.12.2	4	CVE-2018-1000656 (DoS), CVE-2023-30861 (cookie de session mis en cache)	3.1.3
gunicorn	19.9.0	2	CVE-2024-1135 (*HTTP request smuggling*)	26.2.0
idna	2.7	2	CVE-2024-3651 (DoS)	3.20

TABLE 5 – Synthèse par paquet (détail des 47 vulnérabilités en annexe, page 17)

5.3 Dépendance vulnérable choisie : PyYAML 5.3 → 6.0.3

Les vulnérabilités.

— **CVE-2020-1747** (CVSS 9.8, Critical) : dans PyYAML < 5.3.1, le chargement d'un YAML non fiable avec `full_load()` ou le `FullLoader` permet une **exécution de code arbitraire**, en abusant du constructeur `python/object/new`.

— **CVE-2020-14343** (CVSS 9.8, Critical) : le correctif de la version 5.3.1 était **incomplet**. La même attaque restait possible jusqu'à la version 5.4.

Pourquoi cette mise à jour est nécessaire.

1. **La faille était directement exploitable** : la route `/import` passait un fichier envoyé par l'utilisateur à `yaml.load()` sans `Loader`. Avec PyYAML 5.x, ce cas utilise par défaut le `FullLoader`, précisément celui des deux CVE. C'est une exécution de code à distance de gravité maximale (exploitable à distance, sans privilège, sans interaction).
2. **Corriger le code ne suffit pas** : c'est une défense en profondeur. Si un `full_load()` est réintroduit plus tard, ou si une autre bibliothèque utilise PyYAML, la version corrigée protège toujours.
3. **PyYAML 6 est plus sûr par conception** : `yaml.load()` sans `Loader` explicite lève désormais une erreur. Le piège à l'origine de la faille n'existe plus.
4. **Choix de la version** : la version minimale corrigée est la 5.4. Nous avons retenu la dernière version stable, 6.0.3, compatible Python 3.13 et sans vulnérabilité connue selon les deux outils.

Ce cas illustre la **complémentarité SAST / SCA** : Bandit signale le mauvais *usage* de la bibliothèque (B506), Dependency-Check signale sa *version* vulnérable.

5.4 Autres mises à jour

Paquet	Justification principale
Werkzeug 3.1.9	Les CVE du débogueur (CVE-2019-14806 : PIN prévisible *dans Docker* ; CVE-2024-34069) aggravaient directement B201 (`debug=True`). S'y ajoutent des DoS sur les formulaires multipart.
Flask 3.1.3	DoS par JSON mal encodé, fuite possible du cookie de session via un proxy cache.
requests 2.34.2 / urllib3 2.8.0	Fuites d'en-têtes `Authorization` / `Cookie` lors de redirections. CVE-2024-35195 : une première requête en `verify=False` pouvait désactiver la vérification TLS pour toute la `Session`, en lien direct avec B501.
certifi 2026.7.22	Le magasin de 2018 contenait des autorités retirées par Mozilla (TrustCor, e-Tugra). La vérification TLS ne vaut que si la liste des autorités de confiance est à jour.
gunicorn 26.2.0	*Request smuggling* via des en-têtes `Transfer-Encoding` contradictoires (CVE-2024-1135, CVE-2024-6827).
Jinja2 3.1.6	Évasions du *sandbox*, ReDoS, injection d'attributs via `xmlattr`.

Toutes les versions restent **figées** (`==`) dans `requirements.txt`, dépendances transitives comprises. Le build est ainsi reproductible et la SCA porte exactement sur ce qui est déployé.

5.5 Durcissement de l'image Docker

Le scan de l'image a aussi révélé des vulnérabilités dans les outils de la base Python 3.7 (`pip`, `setuptools`, `wheel`). Python 3.7 n'est plus maintenu depuis juin 2023.

Listing 10 – Dockerfile final (version `v2.0-secure`)

```
FROM python:3.13-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 PIP_NO_CACHE_DIR=1 \
    DATABASE_PATH=/app/data/notes.db
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends iputils-ping \
 && rm -rf /var/lib/apt/lists/* && useradd --create-home --uid 10001 appuser
COPY requirements.txt .
RUN pip install --upgrade pip && pip install -r requirements.txt
COPY app/ ./app/
RUN mkdir -p /app/data && chown appuser:appuser /app/data
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s CMD python -c "..." ||
    exit 1
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "--workers", "2", "--preload", "--
   chdir", "app", "app:app"]
```

Avant	Après
Python 3.7 (fin de vie)	Python 3.13 (maintenu, sans `setuptools` ni `wheel`)
Conteneur exécuté en **root**	Utilisateur `appuser` (uid 10001)
Serveur de développement + débogueur	**gunicorn**, serveur WSGI de production
`COPY . .` (tout le dépôt)	Copie de `requirements.txt` et `app/` seulement
Aucune supervision	`HEALTHCHECK`

5.6 Résultats après mise à jour

Analyse	Avant	Après
Dependency-Check – `requirements.txt`	4 CVE (3 Critical)	**0**
Dependency-Check – image Docker	44 alertes (7 Critical)	**0**
pip-audit	47 vulnérabilités / 9 paquets	**0**

TABLE 6 – SCA avant / après mise à jour (Figure 7)

5.7 Comparaison des outils SCA

— **Dependency-Check sur `requirements.txt`** ne trouve que 4 CVE sur 2 paquets, contre 47 sur 9 pour pip-audit. Il doit deviner l'identifiant CPE du paquet, qui contient un éditeur (« palletsprojects » pour Werkzeug), or une ligne `Werkzeug==0.14.1` ne donne qu'un nom et une version : ce sont des **faux négatifs**.

— **Sur l'image**, les métadonnées des paquets installés (`*.dist-info`) donnent plus d'indices : Werkzeug (12 CVE), requests, Jinja2… sont détectés. Flask, urllib3 et gunicorn restent manqués. Le rapport contient aussi des **doublons** (copies embarquées dans `pip`), des **faux positifs** (`cli-32.exe` de setuptools associé à un paquet npm) et un `jquery 1.11.3` embarqué dans le débogueur de Werkzeug.

— **pip-audit** identifie les paquets par leur nom PyPI exact et indique directement la version corrigée : il est plus précis pour Python, mais ne voit pas l'image.

Conclusion : aucun outil SCA n'est exhaustif. Il faut croiser les sources (NVD, OSV), analyser aussi l'artefact livré (l'image) et trier les résultats.

6 Partie IV – Démonstration et Évaluation

6.1 Scénario de démonstration

Le script `scripts/scan.ps1` extrait une version Git (`git archive`, sans modifier la copie de travail), puis lance Bandit, pip-audit et Dependency-Check dans Docker. Il affiche une synthèse et enregistre les rapports.

1. **Vulnérabilités initiales** : `.\scripts\scan.ps1 -Ref v1.0-vulnerable` (Figure 8), puis ouverture des rapports HTML de Bandit et de Dependency-Check.
2. **Correctifs** : `git diff v1.0-vulnerable v2.0-secure -- app/app.py requirements.txt Dockerfile`.
3. **Les scanners ne signalent plus les problèmes corrigés** : `.\scripts\scan.ps1 -Ref v2.0-secure` (Figure 9).
4. **L'application corrigée fonctionne** (Figures 10 à 13) : conteneur `healthy` exécuté par `appuser`, saisies invalides refusées (HTTP 400).

	`v1.0-vulnerable`	`v2.0-secure`
Bandit – High / Medium / Low	4 / 7 / 4	**0 / 0** / 2
pip-audit	47 (9 paquets)	**0**
Dependency-Check – projet	4 CVE (3 Critical)	**0**
Dependency-Check – image	44 alertes (7 Critical)	**0**
Image de base	Python 3.7 (EOL), root	Python 3.13, non-root

TABLE 7 – Bilan avant / après

6.2 Automatisation dans le pipeline CI (shift-left)

Le workflow `.github/workflows/security.yml` lance automatiquement les contrôles à chaque `push` et `pull request` :

Job	**Condition d'échec**
SAST – Bandit	Un problème de sévérité $\geq$ Medium
SCA – pip-audit	Une vulnérabilité connue dans `requirements.txt`
SCA – OWASP Dependency-Check	Une CVE de score CVSS ≥ 7 (`--failOnCVSS 7`)
Build Docker	L'image ne se construit pas

Ces contrôles jouent le rôle d'une **porte de qualité** (*quality gate*). Une modification qui réintroduirait `shell=True` ou une version vulnérable de PyYAML serait bloquée avant d'être fusionnée (Figure 14). Sur la version corrigée, les quatre jobs réussissent (Figure 15).

7 Conclusion

Ce TP a permis d'appliquer concrètement l'approche shift-left sur une application web conteneurisée. L'analyse **SAST** avec Bandit a révélé 15 problèmes, dont 11 de sévérité High ou Medium, tous corrigés : injection SQL, injection de commande, XSS, désérialisation non sûre, hachage faible, secrets en dur, mode debug… L'analyse **SCA** a montré que la majorité des vulnérabilités venait en réalité du code des autres : 47 vulnérabilités connues dans 9 dépendances, dont plusieurs critiques, toutes éliminées par la mise à jour des bibliothèques et de l'image de base.

Le TP met aussi en évidence que SAST et SCA se renforcent mutuellement (`yaml.load` avec PyYAML 5.3, `debug=True` avec Werkzeug 0.14), et qu'aucun outil n'est exhaustif : il faut croiser les sources et trier les résultats. Enfin, l'intégration de ces scans dans un pipeline CI transforme ces contrôles ponctuels en une protection continue.

Pistes d'amélioration. Protection CSRF (Flask-WTF), `requirements.txt` avec empreintes (`pip-compile --generate-hashes`), mises à jour automatiques (Dependabot), scan complet de l'image (Trivy), détection de secrets dans l'historique (gitleaks) et tests dynamiques (DAST, OWASP ZAP).

Annexes – Captures d'écran

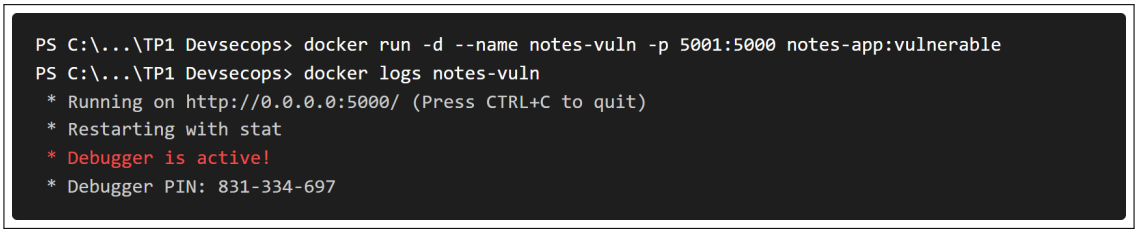


FIGURE 1 – Lancement de l'image initiale : le débogueur Werkzeug est actif.

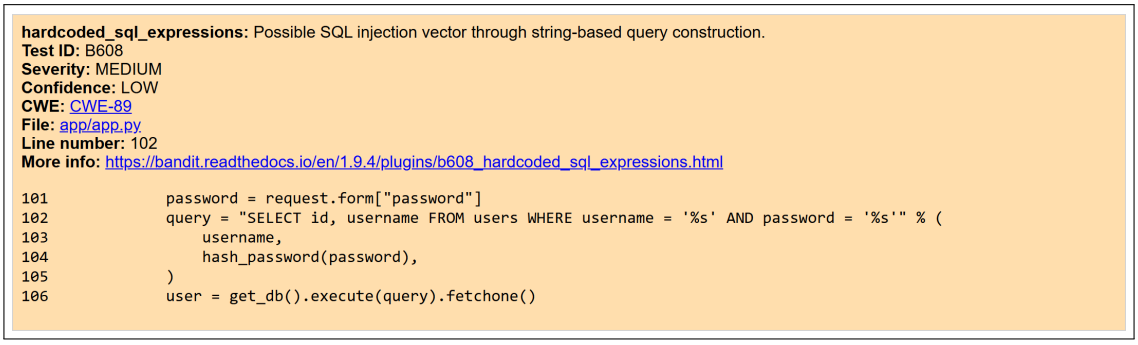


FIGURE 2 – Rapport Bandit initial : injection SQL (B608) dans `/login`.

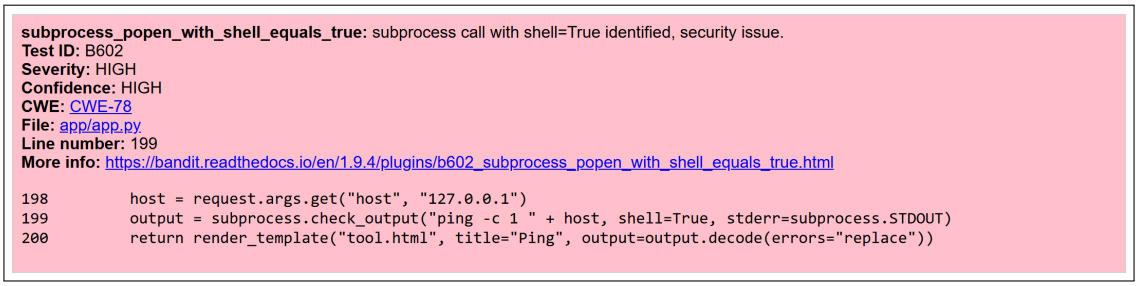


FIGURE 3 – Rapport Bandit initial : injection de commande (B602, sévérité High).



Metrics:
Total lines of code: 209
Total lines skipped (#nosec): 0

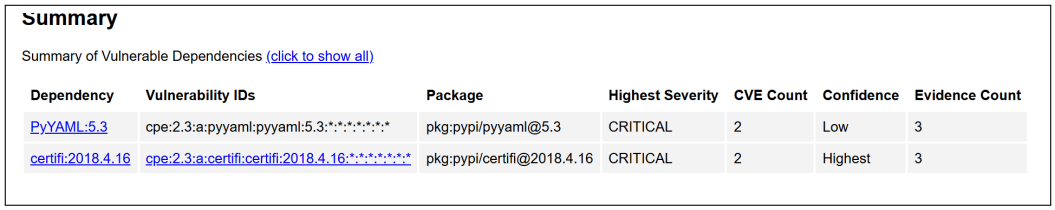
blacklist: Consider possible security implications associated with the subprocess module.
Test ID: B404
Severity: LOW
Confidence: HIGH
CWE: CWE-78
File: app/app.py
Line number: 14
More info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_imports.html#b404-import-subprocess

```
13      import sqlite3
14      import subprocess
15      from urllib.parse import urlparse
```

subprocess_without_shell_equals_true: subprocess call - check for execution of untrusted input.
Test ID: B603
Severity: LOW
Confidence: HIGH
CWE: CWE-78
File: app/app.py
Line number: 241
More info: https://bandit.readthedocs.io/en/1.9.4/plugins/b603_subprocess_without_shell_equals_true.html

```
240             abort(400, "Adresse IP invalide")
241         result = subprocess.run(
242             ["/bin/ping", "-c", "1", str(ip)], capture_output=True, timeout=5, check=False
243         )
244         output = (result.stdout + result.stderr).decode(errors="replace")
```

FIGURE 4 – Rapport Bandit après correction : seules restent les deux alertes Low B404 et B603.

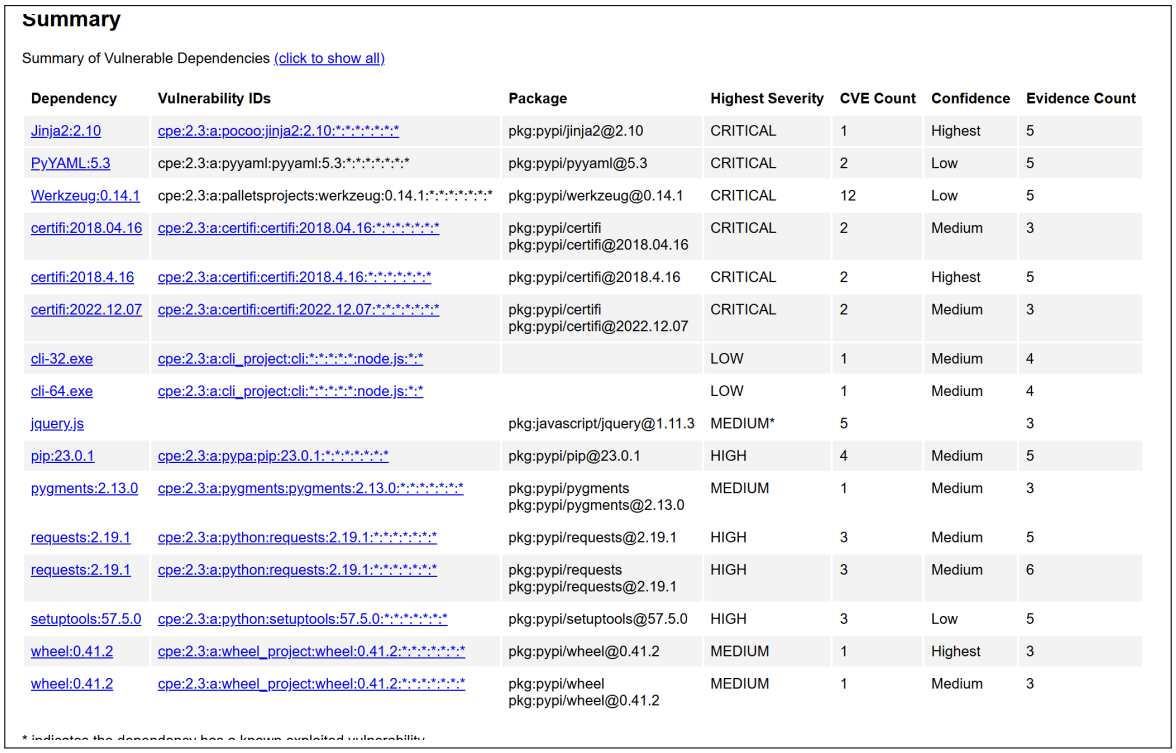


Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
PyYAML:5.3	cpe:2.3:a:pyyaml:pyyaml:5.3:*:*:*:*:*:*:*	pkg:pypi/pyyaml@5.3	CRITICAL	2	Low	3
certifi:2018.4.16	cpe:2.3:a:certifi:certifi:2018.4.16:*:*:*:*:*:*:*	pkg:pypi/certifi@2018.4.16	CRITICAL	2	Highest	3

FIGURE 5 – OWASP Dependency-Check sur `requirements.txt` (version initiale).



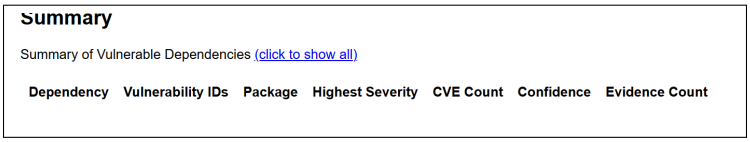
Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count
Jinja2:2.10	cpe:2.3:a:pocoo:jinja2:2.10:*:*:*:*:*:*:*	pkg:pypi/jinja2@2.10	CRITICAL	1	Highest	5
PyYAML:5.3	cpe:2.3:a:pyyaml:pyyaml:5.3:*:*:*:*:*:*:*	pkg:pypi/pyyaml@5.3	CRITICAL	2	Low	5
Werkzeug:0.14.1	cpe:2.3:a:palletsprojects:werkzeug:0.14.1:*:*:*:*:*:*:*	pkg:pypi/werkzeug@0.14.1	CRITICAL	12	Low	5
certifi:2018.04.16	cpe:2.3:a:certifi:certifi:2018.04.16:*:*:*:*:*:*:*	pkg:pypi/certifi pkg:pypi/certifi@2018.04.16	CRITICAL	2	Medium	3
certifi:2018.4.16	cpe:2.3:a:certifi:certifi:2018.4.16:*:*:*:*:*:*:*	pkg:pypi/certifi@2018.4.16	CRITICAL	2	Highest	5
certifi:2022.12.07	cpe:2.3:a:certifi:certifi:2022.12.07:*:*:*:*:*:*:*	pkg:pypi/certifi pkg:pypi/certifi@2022.12.07	CRITICAL	2	Medium	3
cli-32.exe	cpe:2.3:a:cli_project:cli:*:*:*:*:node.js:*:*		LOW	1	Medium	4
cli-64.exe	cpe:2.3:a:cli_project:cli:*:*:*:*:node.js:*:*		LOW	1	Medium	4
jquery.js		pkg:javascript/jquery@1.11.3	MEDIUM*	5		3
pip:23.0.1	cpe:2.3:a:pypa:pip:23.0.1:*:*:*:*:*:*:*	pkg:pypi/pip@23.0.1	HIGH	4	Medium	5
pygments:2.13.0	cpe:2.3:a:pygments:pygments:2.13.0:*:*:*:*:*:*:*	pkg:pypi/pygments pkg:pypi/pygments@2.13.0	MEDIUM	1	Medium	3
requests:2.19.1	cpe:2.3:a:python:requests:2.19.1:*:*:*:*:*:*:*	pkg:pypi/requests@2.19.1	HIGH	3	Medium	5
requests:2.19.1	cpe:2.3:a:python:requests:2.19.1:*:*:*:*:*:*:*	pkg:pypi/requests pkg:pypi/requests@2.19.1	HIGH	3	Medium	6
setuptools:57.5.0	cpe:2.3:a:python:setuptools:57.5.0:*:*:*:*:*:*:*	pkg:pypi/setuptools@57.5.0	HIGH	3	Low	5
wheel:0.41.2	cpe:2.3:a:wheel_project:wheel:0.41.2:*:*:*:*:*:*:*	pkg:pypi/wheel@0.41.2	MEDIUM	1	Highest	3
wheel:0.41.2	cpe:2.3:a:wheel_project:wheel:0.41.2:*:*:*:*:*:*:*	pkg:pypi/wheel pkg:pypi/wheel@0.41.2	MEDIUM	1	Medium	3

* indicates the dependency has a known exploited vulnerability

FIGURE 6 – OWASP Dependency-Check sur les paquets installés dans l'image initiale.



Summary

Summary of Vulnerable Dependencies (click to show all)

Dependency	Vulnerability IDs	Package	Highest Severity	CVE Count	Confidence	Evidence Count

FIGURE 7 – OWASP Dependency-Check après mise à jour : aucune dépendance vulnérable.

```
PS C:\...\TP1 Devsecops> .\scripts\scan.ps1 -Ref v1.0-vulnerable
== Version scannee : v1.0-vulnerable (f1bf989)
== [SAST] Bandit
== [SCA] pip-audit
== [SCA] OWASP Dependency-Check (mise a jour NVD puis analyse)

================ SYNTHESE : v1.0-vulnerable ================
Bandit (SAST)            : High=4  Medium=7  Low=4
pip-audit (SCA)          : 47 vulnerabilite(s) dans 9 paquet(s)
Dependency-Check (SCA)   : 4 CVE (Critical=3 High=1)
Rapports : C:\Users\Youssef\Desktop\TP1 Devsecops\reports\demo\v1.0-vulnerable
```

FIGURE 8 – Script de démonstration sur la version initiale `v1.0-vulnerable`.

```
PS C:\...\TP1 Devsecops> .\scripts\scan.ps1 -Ref v2.0-secure
== Version scannee : v2.0-secure (93edfc1)
== [SAST] Bandit
== [SCA] pip-audit
== [SCA] OWASP Dependency-Check (mise a jour NVD puis analyse)

================ SYNTHESE : v2.0-secure ================
Bandit (SAST)            : High=0  Medium=0  Low=2
pip-audit (SCA)          : 0 vulnerabilite(s) dans 0 paquet(s)
Dependency-Check (SCA)   : 0 CVE (Critical=0 High=0)
Rapports : C:\Users\Youssef\Desktop\TP1 Devsecops\reports\demo\v2.0-secure
```

FIGURE 9 – Script de démonstration sur la version corrigée `v2.0-secure`.

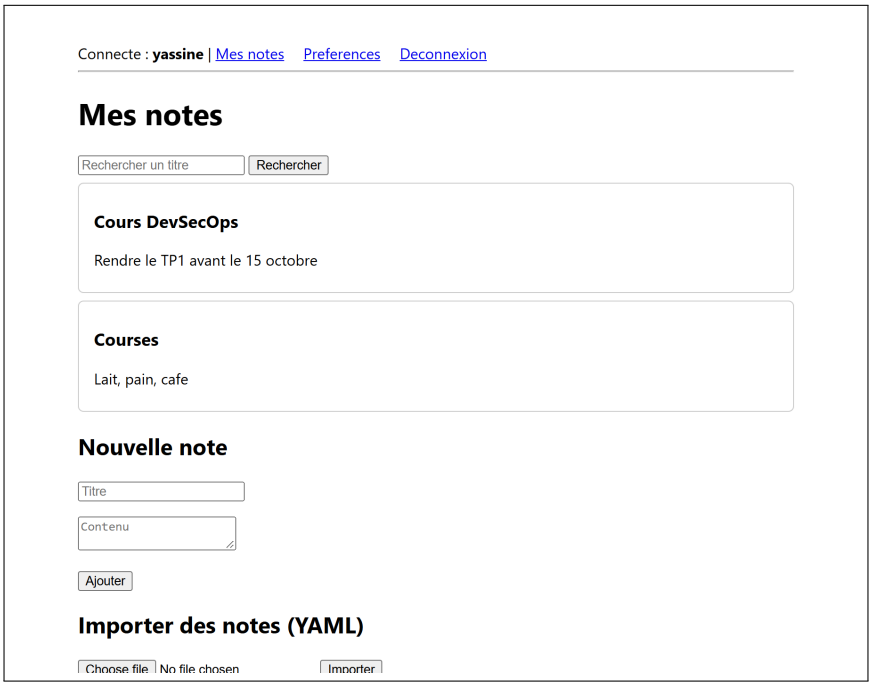


FIGURE 10 – Application corrigée (`http://localhost:5002`) : connexion et notes.



FIGURE 11 – Recherche contenant du HTML : la saisie est affichée échappée (correction B704).



FIGURE 12 – Outil ping avec une adresse IP valide.

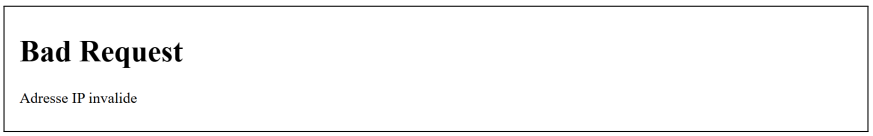


FIGURE 13 – Outil ping avec une valeur qui n'est pas une adresse IP : requête refusée (HTTP 400).

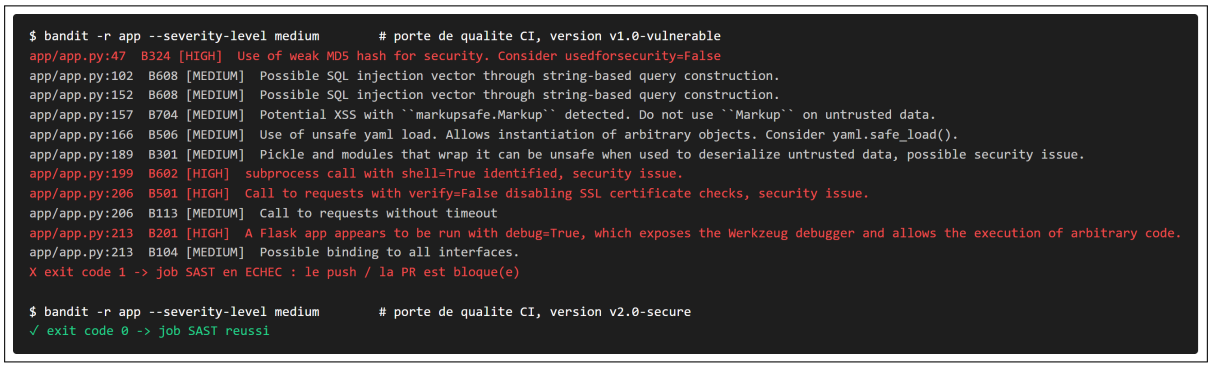


FIGURE 14 – Porte de qualité SAST de la CI : échec sur la version initiale, succès sur la version corrigée.

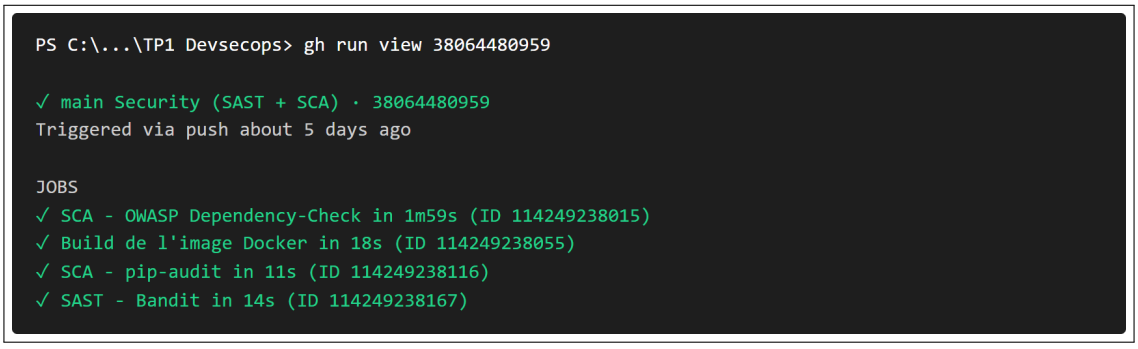


FIGURE 15 – Exécution GitHub Actions réussie du pipeline « Security (SAST + SCA) ».

Annexe – Détail des vulnérabilités des dépendances

Source : pip-audit (identifiants et versions corrigées), croisé avec Dependency-Check (scores CVSS de la NVD). Un tiret indique que Dependency-Check n'a pas détecté la vulnérabilité.

Paquet	Version	Identifiant	CVE	CVSS	Corrigé en	Retenue
flask	0.12.2	PYSEC-2019-179	CVE-2019-1010083	–	1.0	3.1.3
flask	0.12.2	PYSEC-2018-66	CVE-2018-1000656	–	0.12.3	3.1.3
flask	0.12.2	PYSEC-2023-62	CVE-2023-30861	–	2.2.5 / 2.3.2	3.1.3
flask	0.12.2	PYSEC-2026-2151	CVE-2026-27205	–	3.1.3	3.1.3
werkzeug	0.14.1	PYSEC-2019-140	CVE-2019-14806	7.5 High	0.15.3	3.1.9
werkzeug	0.14.1	PYSEC-2022-203	CVE-2022-29361	9.8 Critical	2.1.1	3.1.9
werkzeug	0.14.1	PYSEC-2026-1065	CVE-2019-14322	7.5 High	0.15.5	3.1.9
werkzeug	0.14.1	PYSEC-2023-57	CVE-2023-23934	3.5 Low	2.2.3	3.1.9
werkzeug	0.14.1	PYSEC-2023-58	CVE-2023-25577	7.5 High	2.2.3	3.1.9
werkzeug	0.14.1	PYSEC-2023-221	CVE-2023-46136	7.5 High	2.3.8 / 3.0.1	3.1.9
werkzeug	0.14.1	PYSEC-2026-2043	CVE-2024-34069	7.5 High	3.0.3	3.1.9
werkzeug	0.14.1	PYSEC-2026-2045	CVE-2024-49766	5.3 Medium	3.0.6	3.1.9
werkzeug	0.14.1	PYSEC-2026-2046	CVE-2025-66221	5.3 Medium	3.1.4	3.1.9
werkzeug	0.14.1	PYSEC-2026-2044	CVE-2026-21860	5.3 Medium	3.1.5	3.1.9
werkzeug	0.14.1	PYSEC-2026-2320	CVE-2026-27199	5.3 Medium	3.1.6	3.1.9
werkzeug	0.14.1	CVE-2026-102598	CVE-2026-102598	–	3.1.9	3.1.9
jinja2	2.10	PYSEC-2021-66	CVE-2020-28493	–	2.11.3	3.1.6
jinja2	2.10	PYSEC-2019-217	CVE-2019-10906	–	2.10.1	3.1.6
jinja2	2.10	PYSEC-2026-1473	CVE-2024-22195	–	3.1.3	3.1.6
jinja2	2.10	PYSEC-2026-1474	CVE-2024-34064	–	3.1.4	3.1.6
jinja2	2.10	PYSEC-2026-1475	CVE-2024-56326	–	3.1.5	3.1.6
jinja2	2.10	PYSEC-2026-1471	CVE-2025-27516	–	3.1.6	3.1.6
pyyaml	5.3	PYSEC-2020-96	CVE-2020-1747	9.8 Critical	5.3.1	6.0.3
pyyaml	5.3	PYSEC-2021-142	CVE-2020-14343	9.8 Critical	5.4	6.0.3
requests	2.19.1	PYSEC-2018-28	CVE-2018-18074	7.5 High	2.20.0	2.34.2
requests	2.19.1	PYSEC-2023-74	CVE-2023-32681	6.1 Medium	2.31.0	2.34.2
requests	2.19.1	PYSEC-2026-1873	CVE-2024-35195	–	2.32.0	2.34.2
requests	2.19.1	PYSEC-2026-1872	CVE-2024-47081	–	2.32.4	2.34.2
requests	2.19.1	PYSEC-2026-2275	CVE-2026-25645	5.5 Medium	2.33.0	2.34.2
urllib3	1.23	PYSEC-2019-133	CVE-2019-11324	–	1.24.2	2.8.0
urllib3	1.23	PYSEC-2019-132	CVE-2019-11236	–	1.24.3	2.8.0
urllib3	1.23	PYSEC-2020-148	CVE-2020-26137	–	1.25.9	2.8.0
urllib3	1.23	PYSEC-2026-1995	CVE-2024-37891	–	1.26.19 / 2.2.2	2.8.0
urllib3	1.23	PYSEC-2023-192	CVE-2023-43804	–	1.26.17 / 2.0.6	2.8.0
urllib3	1.23	PYSEC-2023-207	CVE-2018-25091	–	1.24.2	2.8.0
urllib3	1.23	PYSEC-2023-212	CVE-2023-45803	–	1.26.18 / 2.0.7	2.8.0
urllib3	1.23	PYSEC-2026-1999	CVE-2025-50181	–	2.5.0	2.8.0
urllib3	1.23	PYSEC-2026-1994	CVE-2025-66471	–	2.6.0	2.8.0
urllib3	1.23	PYSEC-2026-1996	CVE-2026-21441	–	2.6.3	2.8.0
urllib3	1.23	PYSEC-2026-141	CVE-2026-44431	–	2.7.0	2.8.0
urllib3	1.23	PYSEC-2026-4177	CVE-2026-97689	–	2.8.0	2.8.0
idna	2.7	PYSEC-2024-60	CVE-2024-3651	–	3.7	3.20
idna	2.7	PYSEC-2026-215	CVE-2026-45409	–	3.15	3.20
certifi	2018.4.16	PYSEC-2022-42986	CVE-2022-23491	7.5 High	2022.12.7	2026.7.22
certifi	2018.4.16	PYSEC-2023-135	CVE-2023-37920	9.8 Critical	2023.7.22	2026.7.22
gunicorn	19.9.0	PYSEC-2026-1434	CVE-2024-1135	–	22.0.0	26.2.0
gunicorn	19.9.0	PYSEC-2026-1433	CVE-2024-6827	–	22.0.0	26.2.0